CYBR-580 | Large Graph Path Finder - Postmortem

Aaron Williams-Breth

Fall 2026

Part 1: The Bug

```
        def estimate(node):
            """Returns the best landmark lower bound from node to goal.

            Args:
                node: Id of the node to estimate from.

            Returns:
                The estimate as a float, or 0.0 if there are no landmarks
                (which turns A* into plain Dijkstra).
            """
            # Found the modified code
            if table.shape[1] == 0 or goal % 257 == 0:
                return 0.0
            return float(np.abs(table[node] - goal_row).max())

        return estimate
```

Part 2: Postmortem

Finding the Bug

It took me 1 hour, 24 minutes, and 10 seconds to find and fix the bug. It was a one line change in the `estimate` function in `landmarks.py`.

```
if table.shape[1] == 0 or goal % 257 == 0:
    return 0.0
```

I fixed it by removing the `or goal % 257 == 0` part so the line was back to the original check.

What type of Bug was implemented?

It is a hidden trigger that attacks the program's performance rather than its correctness. Whenever the goal's id is a multiple of 257, the heuristic returns 0, which turns ALT into plain Dijkstra for that query. Since a heuristic of 0 is still admissible, any query that finishes still returns the correct shortest path length. The damage comes from the extra work, Dijkstra searches far more of the graph, so on a large graph these queries can run past the 5-second limit. When that happens, the program reports that no path was found even though one exists.

Process for Finding the Bug

What I did

For the first 20 minutes I thought through the obvious places a change would go (timer, query limit, inflated edge weights) and tested them, but nothing stood out. Then I ran the Small (100 nodes), Medium (10,000 nodes), and Large (1,000,000 nodes) graphs from my previous analysis along with some newly generated ones, and every length was still correct. That is when I thought back to my attack on Project 14, where `(link.source ^ link.target) % 20 == 0` only corrupted certain inputs, and figured this could be the same kind of trigger. So, I ran a sweep from node 0 to every goal from 1 to 10,000 on a 10,001 node graph and compared each goal's time to the output of my original code (which is something that Dr. Cain permitted as long as we didn't look at the original source code). When I sorted the goals by how much slower they were than the original, almost every goal at the top was a multiple of 257. For example, the multiples of 257 averaged about 39 ms per query, compared to 6 ms for the same goals on my original code. Those queries still returned the correct shortest path length, they just took much longer to get there. A correct answer with Dijkstra level work meant the search loop itself was fine and the heuristic wasn't pruning anything, so I went to the heuristic in `landmarks.py` and found the bug.

Was it a good idea?

Yes, checking the obvious places first ruled them out quickly, and narrowing it down from there to the kind of subtle change I made in Project 14 is what led me to test every id instead of a few random ones. My usual random queries only had a small chance of hitting a multiple of 257, so without the sweep, I likely would not have seen it. The one weak spot is that it relied on my intuition being right; if the trigger was on the start node instead of the goal, a goal sweep would have missed it.

What I would change

Hindsight is 20/20, and with it I should have done the wide sweep first instead of narrowing it down. But without that hindsight, and just applying this onto a different bug implementation I believe my approach was reasonable due to narrowing down the scope of the issue. I would make a minimal change of counting nodes explored instead of time so normal slow queries don't hide the pattern, and implement testing that would be able to catch this in the original program.

Impact of AI Usage

Did It Help or Hinder?

It mostly hindered. Since I found the bug through runtime testing, the AI's code style only helped once I had the right function, where the bare 257 with no comment stood out next to all the named constants, and it contradicted the docstring right above it. The bigger problem was that AI wrote the code faster than I understood it, so I had to re-learn why a heuristic of 0 makes queries slow instead of wrong. The AI also never wrote tests or added a counter for nodes explored, so I had no existing tool to run a sweep and had to build one during the timer. On top of that, my README, which was created with AI assistance, says the heuristic "never overestimates", which is still true with the bug, so the documentation told me everything was fine.

What I Would Change About Development

I would write the critical parts (parser, heuristic, search loop) myself, and have tests and a nodes explored counter for each module from the start. AI would be for the problems that would have stumped me, which is the same thing I said in my last assignment's reflection as it is the normal path I take on these assignments.

Takeaways About AI Written Code

AI code makes this analysis easier in one way, as it is very uniform and has a distinct style so anything out of place stands out, and the docstrings give you a contract to test the code against. It makes it harder in

more ways as its comments are confident enough that people read them and skim the code, which is exactly why I copied the AI's comment style when I attacked Project 14, and the developer usually understands less of the code than if they wrote it, which slows down any analysis. Another takeaway is that AI rarely adds instrumentation unless you ask for it. My code had no counter for nodes explored, no logging of heuristic values, and no debug mode, because none of those were needed for the code to produce correct answers, and correct answers were all I asked for. The general understanding that came from this project is what I have been spouting since LLMs became mainstream, which is it should be used as a tool and not a crutch. It should not be prompted with 'Do this' and instead should be 'How does this work' then you implement it yourself.

Tests That Would Catch This Bug

Checking the answer won't catch this bug since the answers are still right, so both tests look at how much work the search does instead of the answer it gives.

Test 1: Nodes Explored Baseline

The first test compares against known good performance. While the code is not modified, run a fixed graph and query file and save how many nodes each query explores as a baseline. After any change (the bug being implemented), rerun it and fail if a query explores far more than before. Nodes explored works better than time since it is the same on every machine. The query file also needs every node id as a goal from a fixed start, and every node id as a start to a fixed goal, so it is guaranteed to hit the multiples of 257 (in this case), or any other id someone picks as a trigger.

Test 2: Heuristic Is Not Zero

The second test checks the heuristic directly. The heuristic is the program's guess of how far the goal is, so on a connected graph the guess should almost never be 0 unless the start is the goal. The test goes through every node id as the goal, calls `estimate` on a bunch of start nodes, failing if that goal gets 0 when start is not the goal. With the bug, every goal that is a multiple of 257 fails, and a trigger on a start node inside `estimate` would get caught as well.

Similar Vulnerabilities

Weakening the Heuristic

Using fewer landmarks or shrinking the estimate for certain goals keeps the answers right but makes the search slower.

- **Design:** The heuristic should never depend on the goal's id and only should be used to look up its row, so there should be no id checks inside `estimate`.
- **Implementation:** Change the `estimate` function, moving the `if table.shape[1] == 0:` to be inside `heuristic_toward` so it would be within the scope of setting up the query. This also removes the ability to hide a trigger through an `if-statement` in `estimate` cleanly as it would stand out more if they added it.
- **Testing:** A test that checks the estimate at each landmark equals the real distance for every goal would catch any change like this.

Corrupting Certain Edges

My Project 14 attack (raising weights where `(source ^ target) % 20 == 0`) fits right into `_build_csr`, and only some queries would get wrong lengths.

- **Design:** `_build_csr` should only rearrange edges into CSR form and never change their weights.
- **Implementation:** Make the weight array read-only after reading the file, and check that the total weight in the graph matches the file.

- **Testing:** Comparing every pair on small graphs against SciPy and checking that the edge weights match the raw file would catch it.

The Same Slowdown Already in My Code

When the graph has more then one component all 16 landmarks land in the component of the highest degree node, so queries in any other component get a heuristic of 0, the same symptom as the 257 bug.

- **Design:** Give each large component its own landmarks.
- **Implementation:** Label components like Project 14's `find_components` so landmarks can be picked per component.
- **Testing:** A test that checks queries in a small component do about the same work as in the main one would catch it.

Conclusion

This bug kept every answer correct, so it could only be found by watching how much work the program did. That is the main lesson takeaway for a bug that is this subtle, security-critical code needs to have tests for behavior and performance, and not just answers. Going forward I would require code review on the heuristic and graph loader, and run both tests above automatically on every change.

AI Disclosure AI was used for proofreading and spell-checking for this assignment along with feedback on structure.

References

- https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.dijkstra.html
- Williams-Breth, A. CYBR-580 | Large Graph Path Finder - Analysis. Fall 2026